

C47/R47 Solves, integration, plotting of RPN functions

EQN menu: algebraic solves, integrals and plots:

The EQN menu allows you to enter formulas in algebraic entry, similar to paper, and the calculator automatically parses and displays the variables.

This way is not handled in this appnote!

ADV menu: RPN solves, integrals and plots

The RPN programmed function follows the traditional HP-42S approach, allowing you to write a short RPN program that evaluates a function or expression step by step on the stack. It supports loops, stores, recalls, advanced functions, sub-functions, and explicitly declares each variable used by the expression with MVAR.

This route offers full control over computation, and this appnote explains the RPN approach in detail.

By hand, from the UI

- Run SOLVE, $\int$ and PLTf on one worked example, straight from the menus.

In an RPN program

- Automate and sequence solve, integrate and plot, then run them together.

Embedded integrals and solves in an RPN program

- Plot the result of a series of integrals; solve for where the integral crosses zero.
- Ten examples of programmed solves and embedded solves/integrals.

The RPN programming techniques behind the demo

- Looping, indirect execution and output formatting used by the demo program.

The 'Gauntlet of Integrals': demo, benchmark and comparison on hardware

- Eleven hard integrals from Duncan Murray's article, implemented as RPN programs,
- with the plots and integral accuracy compared against other calculators.

Table of Contents

C47/R47 Solves, integration, plotting of RPN functions	1
By hand, from the UI	1
In an RPN program	1
Embedded integrals and solves in an RPN program.....	1
The RPN programming techniques behind the demo	1
The 'Gauntlet of Integrals': demo, benchmark and comparison on hardware	1
Table of Contents	2
How does it work ? (SOLVE, INT, PLOT)	5
Manually solve, integrate and plot on C47/R47	6
Step -1: What is MVAR?	6
Step 0: Start with an RPN equation.....	6
Step 1: RPN function/program.....	6
Step 2: SOLVE: Manual solve for the roots.....	7
Step 3: $\int f(x) dx$: Manual integration to get the surface area.....	7
Step 4: PLOTf: Manual graph/plot of MVAR RPN program/formula.....	7
Programmed roots, integrals and graphs - Intro	8
Programmed Roots	8
Programmed integration	8
Minimal program using an RPN function for roots/integrals and graphs.....	9
Programmed plotting of graphs	9
Programmed roots, integrals and graphs – Advanced.....	11
Embedding an integral	11
Nesting.....	11
What you can put inside what.....	11
These combinations work.....	11
Three rules when embedding on the C47/R47	12
What is not supported.....	12
Accuracy, in plain terms	12
Why the answer is always an estimate.....	12
The three things you can set.....	12
The starting values (defaults).....	13

SDIGS, up close	13
Worked example: what if I set SDIGS to 20?	14
ACC, up close.....	14
The automatic graph speed-up.....	15
Accuracy when you nest.....	15
Quick reference	16
RPN programming techniques often used for graphing.....	17
Display Temporary Information with output (AVIEW)	17
Looping with a counter (DSZ / GTO).....	18
Indirect execution (run-time label names)	18
Output formatting ( xy).....	19
Run a program (XEQ)	19
Automated integrals and speed/accuracy testing.....	20
Evaluating the results.....	20
Test the 11 integrals P01 – P11 referred to in the SM Forum.....	20
47 Numerical Results: 47 & Simulator 00.109.03.02b, dd. 1 May 2025, cdaa2bf.....	23
Programmed examples of embedded tests	25
1. PLOT of a SOLVE: the root locus (PLTROOT)	26
2. PLOT of an INTEGRAL, fixed limits: a straight line (PLTINTG)	27
3. PLOT of an INTEGRAL, moving upper limit: a cubic (PLTINT3)	28
4. An INTEGRAL inside an INTEGRAL: the double integral (DBLINT)	28
5. Three integrals deep: the coupled triple integral (TRPINT).....	29
6. PLOT of a double integral (PLTDBL)	30
7. SOLVE of an INTEGRAL: the classic (SLVINT).....	31
8. SOLVE of a SOLVE: a solve inside a solve (SLVSLV)	32
9. Standalone SOLVE at full precision: $\sqrt{2}$ (SLVF2).....	33
10. Standalone INTEGRAL at full precision: π (INTPI)	34
Closing Notes.....	36
Annex A: R47 rejig Program Listing.....	37
Annex B: Alternative graphing: PLSTAT	38
Plot (x, y) graphs using the STAT and PLOT menus.....	38
Graph an RPN function using PLSTAT:.....	38
Graphing the prior RPN function 'FUNC' using the STAT and PLOT menus.....	39

Plot it	40
Annex C: Controlling graph parameters	42
Comparative results: Source Duncan Murray, SwissMicros Forum	43

How does it work ? (SOLVE, INT, PLOT)

Every one of these tools does the same simple thing, over and over: it puts one value (x) into a function f(x) and reads one value out. And then does something with it.

The solver is a good example. To find where your function equals zero, *it tries a value, runs your function once, and reads a single number back. Then it tries another value, a little closer to the answer, and runs again.* Trying, running, reading: that loop is the whole of solving, hence originally called a SOLVER. The grapher does the same to draw a curve on a cartesian plane. For each point across the x-axis on the screen it runs your function once and reads one height, y.

An integral is the third tool, and it asks a different kind of question. *The solver and the grapher each call your function many times over, reading one number at each call.* A single integral instead poses one question and answers it in a single run: what is the net signed area under the curve f(x) between a lower limit a and an upper limit b, written $\int_a^b f(x) dx$? You give it those two limits and point it at the program for f(x), and it works the area out for you. To do that it samples the function at many well-chosen points between the limits and combines them, packing more of its samples near the two ends of the interval than in the middle, rather than stepping evenly, and never landing on the exact endpoints where a function can misbehave. When it finishes it leaves two numbers on the stack: the area, and an estimate of how far off that area might be. That single returned area is the whole output of an integral, and it is exactly what lets an integral, in turn, sit inside another tool.

Manually solve, integrate and plot on C47/R47

We immediately jump into the easiest right in the beginning. Below a few steps to get going quick:

Step -1: What is MVAR?

MVAR stands for “menu variable.” With an MVAR ‘x’ instruction in a program, you are telling the calculator that your RPN program will need a value for your variable used, say x. When you run SOLVE, PLTf, or $\int dx$, the C47/R47 (referred to as 47 below) then shows a variable menu with each declared MVAR variable. You press a number and a function key to store into that variable, or to recall what is currently held there with RCL.

Step 0: Start with an RPN equation

$$f(x) = x^2 + 2x - 2 = 0$$

Create an RPN program that evaluates the expression f(x) and then would solve, integrate or plot for f(x).

Step 1: RPN function/program

The RPN program referred to here is the formula itself. It declares variables using MVAR and uses RCL to place the required variable on the stack for evaluating the expression. This approach allows you to include any variables you want to appear in the solve menu, but it must include at least the variable to be solved for.

C47 Program file export: Export format version 3, C47 program version 1.

0000: Prgm #2/2: 34 bytes / 13 steps

```
0001: LBL 'FUNC'
0002: MVAR 'x'
0003: RCL 'x'
0004: ENTER
0005: x²
0006: x↔y
0007: 2
0008: ×
0009: +
0010: 2
0011: -
0012: END
```

Entering the program:

From run mode, press **XEQ ..** to open a new program space, then f **PRGM** to enter program mode. After entering the program, press f **PRGM** again to return to run mode.

Pro tip: from Didier: To save time, step space and execution time of 2 steps per iteration, use Horner's method for polynomial evaluation to re-arrange the equation to:
 $f(x) = x(x + 2) - 2 = 0$:

```
LBL 'FUNC'
MVAR 'X' RCL 'X'
ENTER 2 + × 2 -
END
```

rejig func.txt -o func.p47

To enter a typical RPN program for a quadratic expression's roots, type:

Type: XEQ .. PRGM LBL α FUNC MVAR α x ENTER x² x↔y 2 × + 2 -

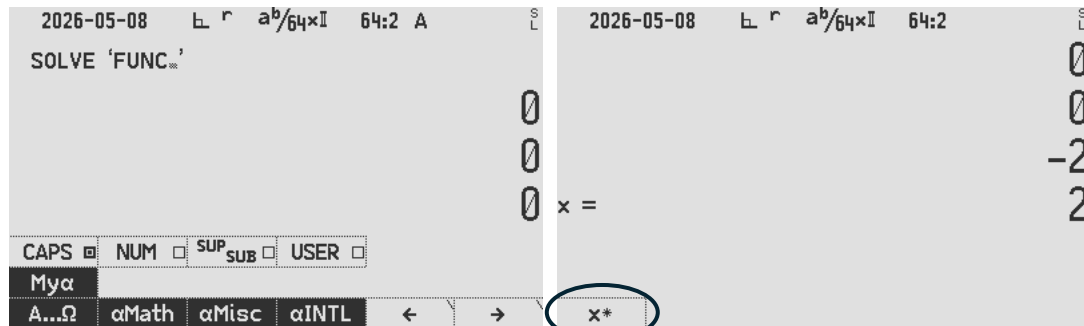
Keystrokes: [XEQ].. f[R/S] [F1] [XEQ] F U N C [F6] [F6] [F4] [XEQ] 1 ENTER f[x²] [x↔y] 2×+2-

- The **MVAR 'x'** command instructs the 47 program to stop and display a menu when SOLVE, **f d** or PLTf is invoked.

- All declared MVAR variables will show in a menu, e.g. 'x' as declared in this example.
- After MVAR 'x' you must use RCL 'x' to use the value on the stack as usual.

Step 2: SOLVE: Manual solve for the roots

- Pick starting points for the solver, for example, set x from -2 to 2 to start the search for roots:



ADV **SOLVE** α **FUNC** **ENTER** 2 **CHS** [x] 2 [x] [x] **STO** 00

Root found, r1: x = 0.732 050 807 568 877 293 527 446 341 505 872 4 stored to R00

Each solution will return only one root. After finding a root, you must provide new starting values to guide the solver towards the next one, similar to the behaviour of the HP 11C, 15C, and 42S. Guide the solver to a different part of the function to search for a second root, e.g. pick more negative points to seek the other root:

10 **CHS** [x] 11 **CHS** [x] [x] **STO** 01

Root r2: x = -2.732 050 807 568 877 293 527 446 341 505 872 stored to R01

Step 3: $\int f dx$: Manual integration to get the surface area

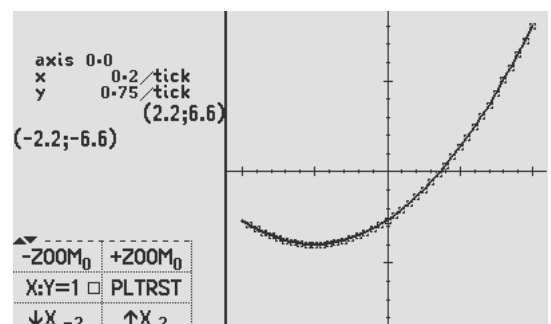
- Get the surface area between the curve and the x-axis, between the two roots:

$\int f dx$ α **FUNC** **ENTER** **RCL** 00 [x] **RCL** 01 [x] [x] $\int_x^y$ **STO** 02

Area: $\int x dx = 6.928 203 230 275 509 174 109 785 366 023 489$ stored to R02

Step 4: PLTf: Manual graph/plot of MVAR RPN program/formula

- Plot a graph of the f(x) where f is the function and x are the MVAR variable.
- **ADV** **PLTf** α **FUNC** **ENTER** 2 **CHS** [x] 2 [x] [x]



Programmed roots, integrals and graphs - Intro

Programmed Roots

Create a program to automate root finding, store them (as above) to R00 & R01:

C47 Program file export: Export format version 3, C47 program version 1.

0000: Prgm #1/1: 146 bytes / 35 steps

```
0001: LBL 'INTDIF'
0002:  PGMSLV 'FUNC'
0003:  -2
0004:  2
0005:  SOLVE 'x'
0006:  STO 00      // store r1
0007:  -11
0008:  -10
0009:  SOLVE 'x'    // store r2
0010:  STO 01
```

Pro tip: Multiple programs

Create the second program within the programming space of the first. This keeps related functions together in one file when saving (WRITEP) or exporting (XPORTP).

Programmed integration

Create a program to find the area between the two roots found above:

```
0011:  PGMINT 'FUNC'
0012:  RCL 00      // lower limit being root1 in Y
0013:  RCL 01      // upper limit being root2 in X
0014:  ∫x 'x'
0015:  STO 02      // store integral
0016:  'x²+2x-2:⇐∫=⇐r₂=⇐r₁='
0017:  STO 04      // 4 lines of display text are stored in r04: Top⇐X⇐Y⇐Z
0018:  CLSTK
0019:  RCL 00      // show root 1
0020:  RCL 01      // show root 2
0021:  RCL 02      // show integral
0022:  AVIEW 04    // show user text
0023:  RTN
```

Greyed out program already entered in the prior sections above:

```
0024: LBL 'FUNC'
0025:  MVAR 'x'
0026:  RCL 'x'
0027:  ENTER
0028:  x²
0029:  x⇐y
0030:  2
0031:  x
0032:  +
0033:  2
0034:  -
0035:  RTN
```

Pro tip: Quick access by typing commands

For quick access to commands, functions, and variables, type α followed by the name directly. For example after **XEQ**, **SOLVE**, **∫f d**, **PLTf**, (marked in red). This is faster than navigating menus, though menu selection works equally well.

Minimal program using an RPN function for roots/integrals and graphs

For a very basic, very short way to program, the MVAR is not required. The basic system fills the stack with X prior to the evaluating function, and this is helpful when a minimal program is written.

Write the function $f(x) = x^2 - 4$ as an ordinary labelled program LBL 'S', x^2 , 4, -, RTN, taking its input from X and leaving the result in X. No MVAR and RCL needed: the plotter fills all four stack levels with X, so use without duplicating with ENTER. Plot from say $x = -1$ to 3, from a second program: LBL 'GO', PGMPLOT 'S', -1 ENTER 3, PLTf 'x', RTN, then XEQ "GO". The variable 'x' is sacrificial in such minimal graph setup, not used; Add SNAP before the RTN to save the plot to a bitmap, or end with PLTFCNS to have the interactive plot menu.

RPN Formula: LBL 'S' x^2 4 - RTN

- LBL 'GO' PGMPLOT 'S' -1 ENTER 3 PLTf 'x' RTN
- LBL 'GO' PGMSLV 'S' 1 ENTER 3 SOLVE 'x' RTN
- LBL 'GO' PGMINT 'S' -1 ENTER 3 $\int_y^x$ 'x' RTN

Compare with algebraic equations:

Plot: LBL 'EQ' 'X^2 - 4' X.SWAP 5 ENTER 9 Draw $\int_y^x$ PLTFCNS SNAP RTN

Complex roots: LBL 'EQ' 'X^2 + 4' X.SWAP 5 ENTER 9 cpxSlv $\int_y^x$ SNAP RTN

Pro tip: Symmetry

Programmed sums and products also fill the stack with X prior to the function:

- LBL 'GO' 1 ENTER 5 ENTER 1 Σ_n 'S' RTN
- LBL 'GO' 1 ENTER 5 ENTER 1 Π_n 'S' RTN

Programmed plotting of graphs

This section deals with the plotting of $y = f(x)$ graphs, defined by your user RPN function FUNC, using the PLTf method:

The MVAR method used is the same as in the solve and integrate functions discussed above.

Create a small program in the same program space as the FUNC program above. Here we continue to enter it from step 36 in the listing above:

```
0036: LBL 'GRAPH'
0037:   PGMPLOT 'FUNC'
0038:   -2
0039:   2
0040:   PLTf 'x'
0041:   SNAP or OPENM 'PLFUNC' for interactive, or PAUSE 20 or PAUSE 99
```

Programmed PLTf plots the graph and stops with the graph visible at the end of the program. If there are more commands after PLTf, it will continue executing without stopping and the options of dealing with the graph are:

- **SNAP** to snap a picture (into a program by typing "XEQ α S N A P ENTER"). (SNAP needs at least a second delay before another SNAP, use PAUSE 10).

- **OPENM 'PLFUNC'** to open the left side menu (into a program by typing "XEQ α 0 P
E N M ENTER α P L F U N C")
- **PAUSE nn**, a short pause to let the user see before continuing.

Programmed roots, integrals and graphs – Advanced

Embedding an integral

When you plot a function as above, or solve for its zero crossings, the tool never needs to know what combination of pieces make up the $f(x)$ program. Again, it is value-in, value-out. So what if that $f(x)$ itself contains a full numerical integral?

When your function contains an integral, the integrator quietly runs its own loop, sampling the integrand at many points, adding them up, and folding all of that into a number at the end of your function. From the outside, nobody sees the loop. We only see one number.

Nesting

The integral's many little steps are tucked inside one step of the solver or the plotter. The solver never sees the busy work; it only ever sees a single value, $f(x)$. Because each tool hides its work behind one number, you can place one tool inside another, and you can do it as deeply as you like.

A picture in words: suppose you want the point x where the area under a curve g first reaches some target K . You ask the solver to make the expression “area from 0 to x , minus K ” equal to zero. At each try, the solver picks an x , the integrator works out the whole area from 0 to that x , K is subtracted, and one number comes back. The solver nudges x up or down until that number is zero. The grapher would draw the same running area against x , one point at a time.

One quiet detail worth knowing: the set-up for the inner job happens inside the run, not before it. Just before your function runs, the trial value is placed in its input variable. Inside the run, the function reads that value, sets the integration limits, points the integrator at the right program, and starts it. The integrator returns its one number, the function finishes, and the calculator tidies up. The only firm rule here is that the variable you integrate over must be a different variable from the one you are solving or plotting.

What you can put inside what

To re-iterate, the solver, the grapher and the integrator each work through your RPN function $f(x)$, or then program, one point at a time, and each keeps its place safely while an inner job runs. That is what lets them nest.

These combinations work

- **Solve an integral.** Find the x for which $\int$ from 0 to x of $g(t) dt$, minus K , equals zero.
- **Plot an integral.** Draw the running area $\int$ from 0 to x of $g(t) dt$ against x .
- **An integral inside an integral.** A double integral, one $\int$ within another.
- **An integral inside an integral inside an integral, etc..** A triple integral, $\iiint$ etc.
- **Plot a root.** The locus of the root, or the value of t that solves $g(t, x) = 0$ at each x .
- **Solve a solve.** An outer equation whose value needs an inner solve to work out.

Three rules when embedding on the C47/R47

- **The inner job must hand back a single number on the stack.** An integral returns its value and a solve returns its root, so each one fits neatly as a term inside an outer function.
- **The inner variable must differ from the outer one.** The solver owns the variable it is solving; the integrator owns the variable it integrates over. They must not be the same.
- **Point the integrator at its program before each nested integral (PGMINT).** The integrator otherwise will not remember its program.

What is not supported

- **An integral whose integrand is itself a solve.** This is unusual mathematically, and is not implemented.
- **Any plot inside the inner function.** A plot draws a picture, and has no single number output, so it cannot serve as a term inside another function.

Accuracy, in plain terms

This is the part most worth your attention. A little understanding here lets you trade speed for precision on purpose, rather than by accident.

Why the answer is always an estimate

A calculator can seldom work out a root or an area exactly; usually it only gets as close as you ask it to, and no closer, though now and then a result lands exactly, as $x = 2$ does in worked Example 7.

Picture the curve of your function as hidden inside a thin ribbon: you tell the calculator how wide that ribbon is, meaning how many significant figures you target, and it returns an answer that fits somewhere inside the ribbon. A narrow ribbon means a careful, slower search; a wide ribbon means a quick, rough one. Asking for more figures than your function really has does not buy you truth, it only costs you time.

The integrator is honest about this. When it finishes, it leaves two things on the stack: the area itself in X, and an estimate of its own uncertainty in Y. The value in Y is the calculator telling you, in effect, “the area is this, give or take about that.” If the give-or-take is larger than you would like, ask for more accuracy and run again.

The three things you can set

Three settings govern accuracy. Two are yours to choose, and one the calculator manages for you.

SDIGS (significant digits). Your main precision dial. It sets how hard the solver works to pin down a root, and how finely the grapher computes each point. It does not affect the integrator.

ACC (accuracy). Your error target for integration, and only for integration. It says how much relative uncertainty to allow in the integrand, which sets the integrator's internal working precision. It works independently of SDIGS.

The graph speed-up (automatic). While a graph is drawing, the calculator quietly lowers its working precision, so the picture appears quickly. You do not set this; at the default it drops SDIGS internally to about 12 significant figures, or to your chosen number when you set SDIGS, in both cases keeping extra guard digits internally, and it never touches the integrator ACC setting.

The starting values (defaults)

Setting	Default	What that means
SDIGS	0 or 34	Work to about 34 significant figures, the full precision of the calculator.
ACC	0	Treated as 1×10^{-32} , an extremely tight target, so an untouched integral is very accurate and in no hurry.
Graph precision	Automatically coupled to SDIGS	SDIGS = 0 gives about 12 digits while a graph draws. Any other value of SDIGS increases or decreases this, for example, SDIGS = 8 is faster but less accurate than SDIGS = 0.

Out of the box, then, the calculator favours precision over speed. That is usually the right choice. The rest of this section is about when, and how, to lean the other way.

SDIGS, up close

SDIGS is the number of significant figures the solver and grapher aim for. At the default, ALL, that is about 34 figures. You can set it lower for speed, or to a fixed number when you want predictable, uniform precision. It keeps a few spare guard figures behind the scenes, so the figures it reports are sound.

Two effects are worth holding in mind together. First, a lower SDIGS lets the solver and grapher finish sooner, at coarser precision. Second, setting SDIGS to a plain number changes how graphs behave. At the default 0 or 34, graphs drop automatically to about 12 figures so they draw quickly. But set SDIGS to, say, 20, and the grapher now works to 20 figures everywhere, including while drawing. The automatic drop to 12 happens only at a setting of 0/34. Regardless of the setting, the grapher keeps a number of internal

guard digits beyond what it reports; for graphs these sit at roughly 3 to 18 digits above the graph working precision, which is enough to keep every graph accurate.

SDIGS setting	Solver and plotter aim for	While a graph draws	Feel
0/34 (default)	about 34 figures	about 12 figures (auto)	Most precise; graphs still brisk
20	20 figures	20 figures	More precise curve, slower graph than 0/34
6	6 figures	6 figures	Coarse and quick; good for a fast operation

Worked example: what if I set SDIGS to 20?

Say you set SDIGS to 20. Here is what changes, step by step.

The solver. It now pins a root to about 20 significant figures instead of 34. That is still far beyond everyday needs, and it finishes a little sooner.

The grapher. It computes each plotted point to 20 figures. Because the automatic drop to 12 applies only at ALL (0/34), your graphs now run at 20 figures throughout, so they draw more precisely, but more slowly than they did at the default.

The integrator. It does not change at all. It follows ACC, not SDIGS, so your integrals keep whatever accuracy ACC asks for.

The screen. How many digits appear on the display is a separate setting. SDIGS is about how hard the calculator works inside, not how many figures it prints.

So “set SDIGS to 20” mostly means this: tidy, uniform 20-figure precision for solving and plotting, slightly quicker solves, slightly slower graphs than the default, and no change to integrals. If you wanted quicker graphs instead, you would choose a small SDIGS such as 6, or return to ALL to get the automatic speed-up back.

ACC, up close

ACC is your error target for integration, and nothing else uses it. It is a relative error: with ACC set to 0.001, for instance, the integrand is trusted to about three figures, and you are declaring that the integrand itself may be off by up to 0.001 in relative terms at any sample point. Why so different to the SDIGS setting? The reason is historical: specifying a target accuracy on the $\int$ key, and getting an uncertainty back, is a convention that dates to the HP-34C (1979) and W. Kahan's integrator, though the 47 uses a different, double-exponential method underneath.

Smaller ACC means a more accurate, slower integral; larger ACC means a quicker, rougher one. You set only ACC; the integrator works out for itself how much internal precision that target needs.

ACC	Meaning	Integral is
0 (default)	Treated as 1×10^{-32} , the tightest target	Extremely accurate, slowest
1×10^{-6}	Aim for about six good figures	Accurate, moderate speed
0.001	Aim for about three good figures	Rougher, quick

Match ACC to what you actually know. If the constants in your function are themselves good to only three figures, asking the integral for ten figures cannot make the answer truer; it only makes the calculator work longer. **Set ACC no tighter than your inputs deserve, and you get a fair answer quickly.**

Uncertainty is in Y. After an integral, Y tells you how far off the calculator thinks its own answer might be. If that is comfortably smaller than the precision you need, you are done. If not, tighten ACC and run again. Note that if the result is 0 and the last incremental change is also 0 the calculator cannot calculate uncertainty and sets Y to 1, as a way of saying it cannot vouch for the result.

The automatic graph speed-up

Drawing a curve asks for about 40 points (a few more, up to around 80, where a steep slope or an asymptote earns extra sampling), and screen pixels are coarse, so full precision on every point would be wasted effort. To keep graphs quick, the calculator lowers its working precision while it draws, down to about 12 figures at the default SDIGS. This is the one accuracy adjustment it makes on its own.

Two things follow. The speed-up applies to the solver and the grapher, so a root drawn on a graph is found only to the graph's precision, not the full standalone precision. It does not apply to the integrator, which always honours ACC. And, as noted above, the drop to 12 figures happens only at the ALL (0/34) setting; choose a fixed SDIGS and the grapher uses that number instead.

Accuracy when you nest

When one tool runs inside another, accuracy follows a few clear rules.

Solve inside a solve. Both the outer and the inner solve run at set precision. Nesting costs you nothing here; each finds its root to about your SDIGS setting.

An integral inside an integral. Each integral honours the ACC in force when it runs, and you set ACC for each level as you need it. They are not fully independent, though: the outer integral can be no more accurate than the inner one it is built on, because the inner result carries into the outer integrand.

Solve inside a plot. A solve that lives inside a plotted function follows the reduced graph precision, so a root drawn on a graph is found only to the graph's figures, about 12 at the default. This keeps drawing quick and is ample for the screen. Raise SDIGS if you want a more precise curve.

An integral inside a plot. An integral drawn on a graph keeps its set ACC accuracy; the graph's speed-up does not touch it. Each plotted point is worked out to ACC, exactly as it would be on its own. So a tight ACC makes an integral plot slow point by point; loosen ACC when you want the graph to appear sooner.

Quick reference

If you want...	Set this
The safe, accurate default	SDIGS 0/34, ACC 0. Precise and unhurried.
A faster, rougher graph	A small SDIGS such as 6, or stay at ALL for the automatic speed-up.
A quicker integral	A larger ACC such as 0.001, matched to what your inputs deserve.
A more precise curve	A higher SDIGS. Expect slower drawing.
An honest error estimate	Read Y after an integral; it holds the give-or-take.

In short: SDIGS steers the solver and the grapher, ACC steers the integrator, and the calculator quietly trades a little graph precision for speed while it draws. Start from the accurate defaults, and loosen only the setting that is costing you time.

RPN programming techniques often used for graphing

INTDEMO – the demo program that runs the benchmark in the next section – is built from the RPN techniques below.

The 47 program referenced is used as a reference, and can convert to a p47 program file in both directions, using [rejig](#). The program serves as a benchmark of the plotting and integration subsystems across easy and deliberately hard integrands (singularities, power towers, wild oscillation).

INTDEMO loops over eleven test functions, plotting each one and then numerically integrating it over its bounds, printing the integral, its error estimate and a relative-accuracy figure. This demo further exercises the following 47 programming techniques:

- Alpha and numeric labels: (LBL 'INTDEMO', LBL 00)
- Counted loop: (DSZ/GTO (11 down to 1))
- Setting calculator mode: (RAD)
- Building label name at runtime: ('BND_P' + RCL 00),
- Executing indirectly: (XEQ →R02).
- Setting graph & integration limits on the stack from a subroutine.
- Plotting: PGMPLT, PLT f 'x', PLTRST
- Integration: PGMINT, ∫xd
- Output and formatting: xy
- Computing relative accuracy: |x|, log₁₀(x)
- Function and stack use: √x, 1/x, x², e^x, tan, cos, sin, ln, Γ, y^x
- User info: AVIEW

Three of these techniques carry most of the program's structure and reward a closer look with the listing in hand (Annex A): the counted loop, indirect execution, and the formatting of printed output.

Display Temporary Information with output (AVIEW)

2026-03-15 L ° /64x 64:2 S L

$x^2+2x-2:$ $x^2+2x-2:$

$r1=$ 0.732 050 807 568 877 294

$r2=$ -2.732 050 807 568 877 29

$\int=$ 6.928 203 230 275 509 174

INTDIF automated root find and output

Pro tip 3:

Create custom text labels in step 0016 and label the stack elements in the sequence of $\text{Top} \leftarrow X \leftarrow Y \leftarrow Z$ in step 0022 above. The mapping is shown in colours as follows:

$x^2+2x-2: \leftarrow r1 \leftarrow r2 \leftarrow \int$

Looping with a counter (DSZ / GTO)

All eleven test cases are driven by one counted loop. Decrement Skip if Zero (DSZ) is used as one of the anchor loop structures. A counter is initialised once and decremented on each pass:

```
11
STO R00
LBL 00
... loop body ...
DSZ R00
GTO 00
RTN
```

R00 is loaded with 11 and LBL 00 marks the top of the loop. At the bottom, DSZ R00 (“decrement, skip if zero”) subtracts one from R00 and skips the following GTO 00 only when the counter reaches zero; otherwise control jumps back to LBL 00. The body therefore runs eleven times, with R00 counting $11 \rightarrow 1$.

Indirect execution (run-time label names)

Rather than an eleven-way branch, INTDEMO builds the name of the routine it needs at run time and calls it indirectly. On each loop it concatenates a fixed string with the loop counter to form the target label:

```
'BND_P'
RCL R00
+
STO R02
...
XEQ →R02
```

Pushing the alpha string 'BND_P', recalling the counter (say 7) and adding them with + yields the label name “BND_P7”, stored in R02. XEQ →R02 then executes the label whose name is held in R02. The arrow (→) is the indirect prefix, meaning “use the contents of the register, not the register itself.” The integrand routines are dispatched the same way: 'INT_P' + RCL R00 builds INT_P7 into R01, and PGMINT →R01 points the integrator at it. One loop body thus services all eleven bound/integrand pairs (BND_P1...BND_P11, INT_P1...INT_P11) with no branch table.

Output formatting (xy)

Results are written to the print/disk log with the xy command, which outputs a labelled line from the stack. The program interleaves a little arithmetic with string building to format each line:

RCL Z

÷

|x|

$\log_{10}(x)$

CHS

'Relative Accuracy:'

xy

Here the error (in Z) is divided by the result, reduced to a magnitude with |x|, and passed through $\log_{10}$ then CHS – giving $-\log_{10}|\text{error} \div \text{result}|$, effectively the number of correct significant digits.

The alpha string 'Relative Accuracy:' supplies the line label, and xy prints value and label together. Units are tagged the same way elsewhere: a time value is divided by 10 and the literal ' s' is appended with + before printing, so the log reads, e.g., “8.5 s”.

Run a program (XEQ)

Get the result by running the program defined above: type **XEQ** **α** **INTDIF** **ENTER**

Automated integrals and speed/accuracy testing

This section puts the 47's RPN integrator through a published 'Gauntlet' of eleven hard integrals. We first set out how accuracy is judged, then run the eleven problems with the 47's results, and finally compare the 47 against other calculators.

Evaluating the results

To evaluate the results, we use reference data, with more digits and more accuracy than the 34-digit calculator provides.

The accuracy figure is determined by comparing the error with the result, using $-\log_{10}(|\frac{\text{error}}{\text{result}}|)$. The accuracy calculation is done in two ways, namely:

1. using the claimed error and result from the integrator on the calculator, i.e. $-\log_{10}(|\frac{47 \text{ error}}{47 \text{ result}}|)$. This tests the full result that the calculator claims.

2. using the actual error and the actual reference result, i.e.

$$-\log_{10}\left(\left|\frac{47 \text{ result} - \text{reference result}}{\text{reference result}}\right|\right).$$

This tests the accuracy of the actual error, ignoring the Y-value given which is at best an estimate.

The reference integral result values used for the 11 defined problems:

Problem	Reference	Source
P1	0.828 427 124 746 190 097 603 377 448 419 396 157 139 3	Duncan
P2	1.633 031 481 071 948 259 293 952 014 043 698 584 924 1	Duncan
P3	2 259	Duncan
P4	11.970 716 785 493 139 183 256 615 050 324 309 092 447 4	Duncan
P5	0.731 339 779 891 570 998 493 721 841 652 133 420 783	Duncan
P6	-0.946 083 070 367 183 014 941 353 313 823 179 657 812 3	Jaco, Wolframalpha
P7	2	Duncan
P8	1.772 453 850 905 516 027 298 167 483 341 145 182 797 5	Duncan
P9	0.918 938 533 204 672 741 780 329 736 405 617 639 861 3	Duncan
P10	2.996 339 811 976 535 379 182 861 556 235 501 887 668	Duncan
P11	0.390 992 162 151 530 454 531 709 880 804 974 485 310 3	Duncan

Test the 11 integrals P01 – P11 referred to in the SM Forum

Below in Annex A is the code that runs these eleven integrals on the 47, with an analysis of the accuracy of the returned results.

Duncan Murray compiled a list of [numerical integration results of the gauntlet of integrals](#), mostly from Valentin Albillo's various posts of most challenging definite integrals, as discussed on the [SwissMicros Forum](#) thread started by Walter. The reason for this effort is to compare the different algorithms that different calculators use, for example, the 47 calculator employs the 'New DEI' method:

The double-exponential integrator in the 47 implements the improved scheme described by Takahasi and Mori (1974), refined in R. van Engelen's qthsh comparison. This scheme benchmarks several different integrators and outperforms the older Michalski-Mosig approach it replaces. The translation runs in the calculator's extended decimal precision (internally to as many as 75 digits) and can handle finite, semi-infinite, and doubly infinite limits. For semi-infinite integrals, a quick pre-pass identifies an optimal split point before the main quadrature begins, resulting in significant speed-up with minimal additional work.

Below the 11 integrals referred to in [Duncan's list](#), complete with 47 result and 47 plot. The accuracy method and reference values are given under Evaluating the results above; the R47 values were confirmed 2025-06 and 2025-12.

Pnn	Text	Equation	
P01	$\int$ from 1 to 2 of $1/\sqrt{x}$ dx Result: 0.8284271217161000076032771181103084	$\int_1^2 \frac{1}{\sqrt{x}} dx$	
P02	$\int$ from -1 to 3 of $e^{-(x^2)}$ dx Result: 1.623031481071018250203052014013600	$\int_{-1}^3 e^{-x^2} dx$	
P03	$\int$ from -15 to 15 of $ x^2 - 3x $ dx Result: 2259.0022999206902167201072568946154	$\int_{-15}^{15} x^2 - 3x dx$	
P04	$\int$ from 0 to 1.57079 of $\tan(x)$ dx Result: 11.07071678510213018225661505023080	$\int_0^{1.57079} \tan x dx$	
P05	$\int$ from 0 to 1 of $x^{x^{x^4}}$ dx (i.e. $x^{(x^{(x^4)})}$) Result: 0.7212207708015700081027212116521224	$\int_0^1 x^{x^{x^4}} dx$	

Pnn	Text	Equation	
P06	$\int$ from 0 to 1 of $\cos(x) * \ln(x)$ dx Result: $-0.0460830703671820410413523138231706$	$\int_0^1 \cos x \ln x \, dx$	
P07	$\int$ from 0 to 1 of $1/\sqrt{x}$ dx Result: 2	$\int_0^1 \frac{1}{\sqrt{x}} \, dx$	
P08	$\int$ from 0 to 1 of $1/\sqrt{-\ln(x)}$ dx Result: $4.772452850005516043240002002374754$	$\int_0^1 \frac{1}{\sqrt{-\ln x}} \, dx$	
P09	$\int$ from 0 to 1 of $\ln(\Gamma(x))$ dx Result: $-0.0480285222046727447802207264056476$	$\int_0^1 \ln \Gamma(x) \, dx$	
P10	$\int$ from -1 to 1 of $e^{x + \sin(e^{e^{e^{x+1/3}}}})} dx$ Result: $0.60857410600580236632782812721218$	$\int_{-1}^1 e^{x + \sin e^{e^{x + \frac{1}{3}}}} \, dx$	
P11	$\int$ from 0 to 1 of $\sin^2(\tan(\tan(\pi*x))) dx$ Result: $4.000624722002700675800525061652740$	$\int_0^1 \sin^2(\tan \tan \pi x) \, dx$	

Agrees with 2025-12-12 version c9249ddd3 tests by Ralf.

23 Mostert © 2026

Pnn	47-model	47 result	47 claimed accuracy	R47 Time	Self Rel. Error	Ref. Rel. Error
	R47 Hardware			100.9 s		
P11	C47 SIM	0.4009624732903700675809525961653749	0.04947881809553660276917773125497109	6.9 s	0.9	1.6
	R47 Hardware			75.5 s		

Red indicates significant difference in the self-ascertained 'error'.

Self Relative Error

$= -\log_{10}(|\frac{47\ error}{47\ result}|)$

Reference Relative Error

$= -\log_{10}\left(|\frac{47\ result-reference\ result}{reference\ result}|\right),$

Programmed examples of embedded tests

This chapter walks through a C47/R47 test suite, one small program at a time. Each program is real and runnable, and each has an analytical answer that we already know, so we can check the calculator against the truth. For every example below, there are four things: the plain question, the exact formula behind it, the short program that poses it to the calculator, a picture of the actual screen, and a look at the figures that came back.

The programs were run end to end by a RPN sequencer program called ALLTEST, which pauses at each step to screen capture the result. Two kinds of pictures appear: a stack picture (“a”) shows the four display lines X, Y, Z and T, with the answer in X and, for an integral, its uncertainty in Y. A graph picture (“b”) shows the output curve. Alongside the pictures, the calculator also wrote every register out to full width, so the discussion below can quote all thirty-four figures, not just the dozen the screen has room for.

The shared demo function is $f(x) = x^2 - p \cdot x - 2$ with parameter p , and is called in turn by the various calling programs listed below. Every program, exact answer below:

Program	What it computes	The result it shows
1. PLTROOT	Plot the locus of roots of $f(x) = x^2 - p \cdot x - 2$ over p	rising curve (SOLVE in PLOT)
2. PLTINTG	Plotting the result of 40+ integrals $\int_0^8 f \, dx$ over p	line $154.67 - 32p$ (INT in PLOT)
3. PLTINT3	Plotting the result of 40+ integrals $\int_0^u f \, dx$ over u	cubic $\frac{u^3}{3} - 2u$ (INT in PLOT)
4. DBLINT	Double $\int_0^2 \int_0^x t \, dt \, dx$	$\frac{4}{3}$ (INT in INT)
5. TRPINT	Triple $\int_0^2 \int_0^y \int_0^x t \, dt \, dx \, dy$	$\frac{2}{3}$ (INT in INT in INT)
6. PLTDBL	Plot the result of the double integral $\int_0^y (\frac{x^2}{2}) \, dx$ over y	the cubic $\frac{y^3}{6}$ (PLOT of INT in INT)
7. SLVINT	Solve the root of the integral $\int_0^x t \, dt - 2 = 0$	$x = 2$ (SOLVE of INT)
8. SLVSLV	Solve for the p whose root is 2	$p = 1$ (SOLVE of SOLVE)
9. SLVF2	Standard solve $x^2 - 2 = 0$	$\sqrt{2}$, full precision (SOLVE)

10. INTPI	Standard integral $\int_0^1 \frac{4}{1+x^2} dx$	π , full precision (INT)
--------------	---	------------------------------

1. PLOT of a SOLVE: the root locus (PLTROOT)

The question: as the parameter p slides from -5 to 5 , the equation $f(x) = x^2 - p \cdot x - 2 = 0$ has a moving positive root. Where does that root sit at each value of p ? Drawing it gives a root locus, a curve of “the answer” against “the setting”.

The formula: For each fixed p the calculator solves $x^2 - p \cdot x - 2 = 0$ and keeps the root, then plots that root against p . By the quadratic formula the root shown is $x(p) = \frac{p + \sqrt{p^2 + 8}}{2}$, a gently rising curve that never dips to zero because the $\sqrt{p^2 + 8}$ term keeps it positive.

The program FX is the function. RT solves it for one value of p and hands back the root. PLTROOT points the grapher at RT and draws the locus over p .

```

LBL 'FX'                ; f(x) = x^2 - p*x - 2
MVAR 'x'
MVAR 'p'
RCL 'x' ENTER *        ; x^2
RCL 'x' RCL 'p' * -     ; - p*x
2 -                    ; - 2
RTN

LBL 'RT'                ; for fixed p: solve f(x)=0, return the root
PGMSLV 'FX'
0 8                    ; initial guesses lo -> Y, hi -> X
SOLVE 'x'              ; root left in X
RTN

LBL 'PLTROOT'           ; outer: plot the locus over p
PGMPLT 'RT'
-5 5                   ; plot range for p
PLTF 'p'
RTN

```

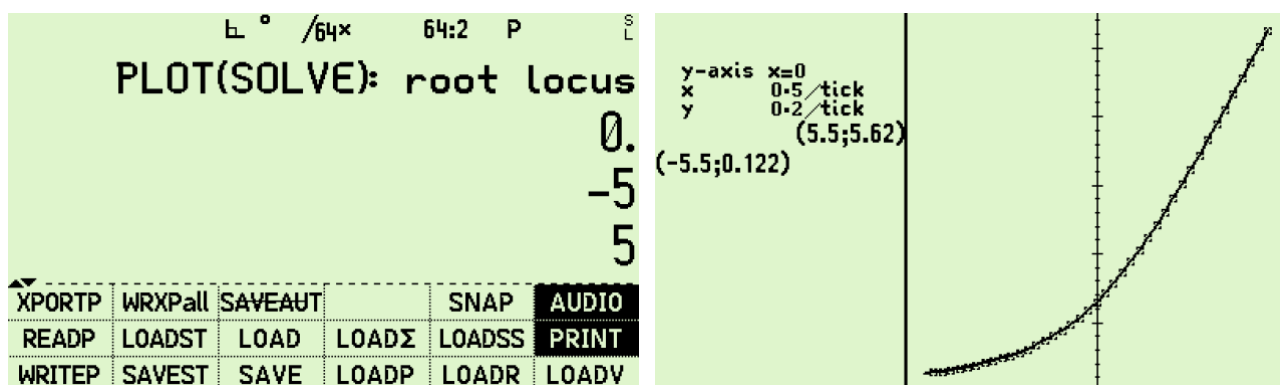


Figure 1a, left. The stack just before the plot. The label sits in T, and the plot range -5 to 5 waits in Y and X.

Figure 2b, right. The root locus itself: the moving positive root of f drawn against p , a smooth rising curve.

The result: The curve rises smoothly and monotonically across the plotted range $p = -5$ to $p = 5$, exactly the shape the quadratic formula predicts (the corner labels at -5.5 and 5.5 mark the graph frame, not points on the curve).

This is a SOLVE living inside a PLOT: at every one of the roughly 40 screen points (more where the slope steepens, up to around 80) the grapher quietly ran the solver to find that point's root. Because the solve happened while a graph was drawing, each root was found to the graph's working precision, about twelve figures at the default, which is far finer than the eye can see and keeps the drawing quick.

2. PLOT of an INTEGRAL, fixed limits: a straight line (PLTINTG)

The question: Fix the limits at 0 and 8 and integrate f over x . The area still depends on p . How does that area change as p slides from -5 to 5 ?

The formula: $\int_0^8 (x^2 - p \cdot x - 2) dx = \frac{x^3}{3} - \frac{px^2}{2} - 2x$ evaluated from 0 to 8 gives $\frac{512}{3} - 32p - 16 = 154.67 - 32p$. The p term is the only one that moves, and it moves in a straight line, so the plot must be a straight line of slope -32 .

The program IG integrates f from 0 to 8 for one p . PLTINTG plots IG over p . FX is the same function as before.

```
LBL 'IG'                ; for fixed p: integrate f over x from 0 to 8
PGMINT 'FX'
0 8                    ; integration limits lower -> Y, upper -> X
∫~x_y 'x'              ; INT(0..8) f dx left in X
RTN

LBL 'PLTINTG'           ; outer: plot the integral over p
PGMPLT 'IG'
-5 5                   ; plot range for p
PLTf 'p'
RTN
```

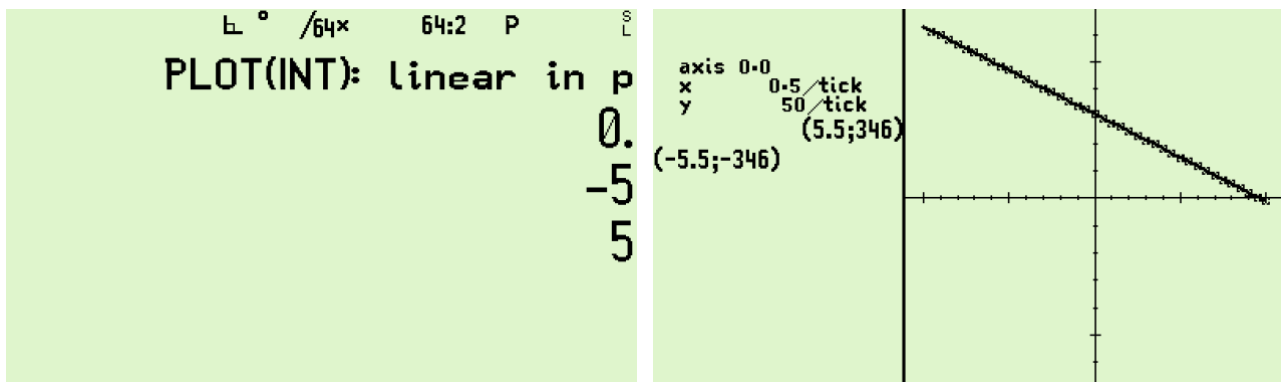


Figure 3a, left. The stack before drawing, with the plot range -5 to 5 in Y and X .

Figure 4b, right. A dead straight descending line: the fixed-limit integral is first order in p , slope -32 .

The result: The screen shows a straight line falling from upper left to lower right, which is what “linear in p ” means. This is an INTEGRAL living inside a PLOT. Note the important accuracy point from earlier: every one of these plotted points was integrated to full accuracy, exactly as if it stood alone as a single integral calculation.

3. PLOT of an INTEGRAL, moving upper limit: a cubic (PLTINT3)

The question: Now let the upper limit itself be the plot variable. Integrate f from 0 up to u , and watch the running area grow as u sweeps from -5 to 5 . Here p is left at its default of 0, which keeps the shape clean.

The formula: With $p = 0$, $\int_0^u (x^2 - 2) dx = \frac{u^3}{3} - 2u$. A running area whose upper limit moves is one order higher than its integrand, so a quadratic integrand gives a cubic curve.

The program IU integrates f from 0 to u . PLTINT3 plots IU over u .

```
LBL 'IU'           ; integrate f over x from 0 to u (the plot variable)
MVAR 'u'
PGMINT 'FX'
0 RCL 'u'          ; limits: lower 0 -> Y, upper u -> X
∫^x_y 'x'          ; INT(0..u) f dx left in X
RTN

LBL 'PLTINT3'      ; outer: plot the running integral over u
PGMPLT 'IU'
-5 5               ; plot range for u
PLTf 'u'
RTN
```

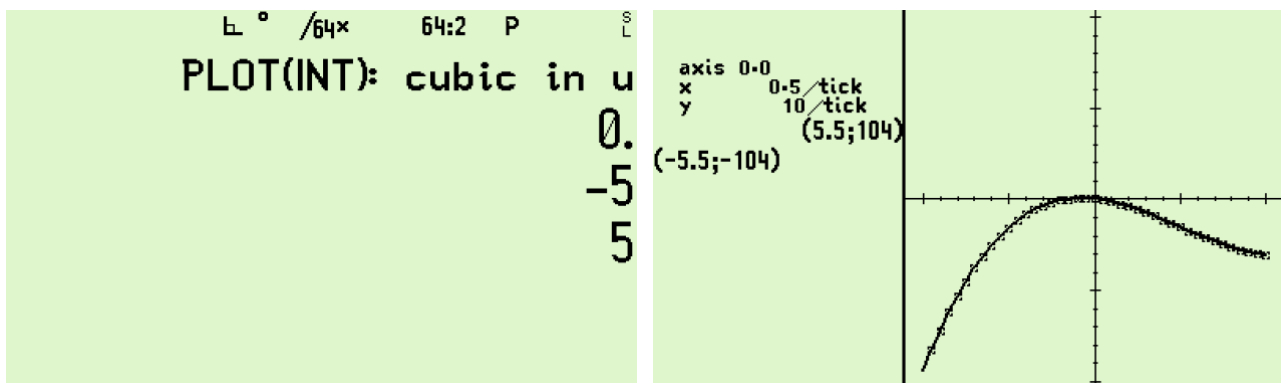


Figure 5a, left. The stack before drawing the running integral.

Figure 6b, right. The cubic $u^3/3 - 2u$: One order higher than the straight line of example 2.

The result: The curve has the unmistakable S-curve of a cubic, with a small hump-and-dip near the middle from the $-2u$ term. The single change from example 2, letting the limit move instead of the parameter, lifts the answer from first order to third order. Same integrator, same function, a very different picture.

4. An INTEGRAL inside an INTEGRAL: the double integral (DBLINT)

The question: Can the calculator work out a double integral, one integral sitting inside another? The test is a case with a tidy known answer.

The formula: $\int_0^2 \int_0^x t dt dx = \int_0^2 \left(\frac{x^2}{2}\right) dx = \frac{4}{3} = 1.33333...$ The inner integral turns t into $\frac{x^2}{2}$; the outer integral then integrates that from 0 to 2.

The program HT is the innermost integrand, just $h(t) = t$. IT integrates it from 0 up to x , giving $\frac{x^2}{2}$. DBLINT integrates IT from 0 to 2.

```

LBL 'HT'                ; innermost integrand h(t) = t
MVAR 't'
RCL 't'
RTN

LBL 'IT'                ; INT(0..x) t dt = x^2/2
MVAR 'x'
PGMINT 'HT'
0 RCL 'x'
∫^x_y 't'              ; inner integral, over t
RTN

LBL 'DBLINT'            ; INT(0..2) IT dx = 4/3
PGMINT 'IT'
0 2
∫^x_y 'x'              ; outer integral, over x
RTN

```

Figure 7a. Result in X, uncertainty in Y. X reads 1.33333... (four thirds); Y is a few parts in 10^{35} .

The result, to full width:

Register	Value as captured (34 significant figures)
X (area)	1.333333333333333333333333333333
Y (uncertainty)	2.565×10^{-35}
Z	1.999999999999999999999999999998 (the outer upper limit, 2)

Two variables were in play at once, x on the outside and t on the inside, and the C47/R47 rule that they must differ is exactly why this works. X holds $\frac{4}{3}$ to the full thirty-four figures. Y is the integrator being honest about itself: it reckons its own error is only a few parts in 10^{35} , which is why we can trust every figure shown.

5. Three integrals deep: the coupled triple integral (TRPINT)

The question: Does the nesting keep working when it goes three levels deep, with each level's limit set by the level above it?

The formula: $\int_0^2 \int_0^y \int_0^x t \, dt \, dx \, dy = \int_0^2 \left(\frac{y^3}{6}\right) dy = \frac{2}{3} = 0.66667\dots$ The three levels are coupled through their limits: t runs up to x, x runs up to y, y runs up to 2. Every level does real work. This does take real time to perform, better use a simulator on a fast computer.

The program IY is the double integral with a moving upper limit y; it reuses HT and IT from example 4. TRPINT integrates IY from 0 to 2.

```

LBL 'IY'                ; INT(0..y) IT dx = y^3/6 (reuses HT, IT)
MVAR 'y'
PGMINT 'IT'
0 RCL 'y'
 $\int^x$  'x'
RTN

LBL 'TRPINT'            ; INT(0..2) IY dy = 2/3
PGMINT 'IY'
0 2
 $\int^y$  'y'
RTN

```

```

L ° /64x 64:2 P S
INT(INT(INT)) = 2/3
1.333 333 333 333 333 333 3
7.412 812 5×10-35
0.666 666 666 666 666 666 7

```

Figure 8a. Three integrals deep. X reads 0.66667..., with the uncertainty in Y and the inner double-integral value in Z.

The result, to full width:

Register	Value as captured (34 significant figures)
X (area)	0.66666666666666666666666666666667
Y (uncertainty)	7.4128125×10 ⁻³⁵
Z	1.3333333333333333333333333333331 (the inner double integral at y = 2)

Three integrals, three different variables (t, x, y), each honouring its own accuracy, and the answer $\frac{2}{3}$ lands to the full thirty-four figures. At the default tight ACC, nesting costs almost nothing in accuracy: each level saved its place on the stack while the level below runs, as deep as the stack allows.

6. PLOT of a double integral (PLTDBL)

The question: Take the inner double integral from example 5, the one that equals $\frac{y^3}{6}$, and instead of evaluating it at a single point, draw it as y sweeps from -5 to 5.

The formula: $IY(y) = \int_0^y \left(\frac{x^2}{2}\right) dx = \frac{y^3}{6}$, a pure cubic through the origin. Warning: this also takes real time to complete. Again a simulator on a fast computer is recommended.

The program PLTDBL plots IY over y, reusing HT, IT and IY.

```
LBL 'PLTDBL'           ; plot IY(y) = y^3/6 over y (reuses HT, IT, IY)
PGMPLT 'IY'
-5 5                   ; plot range for y
PLTF 'y'
RTN
```

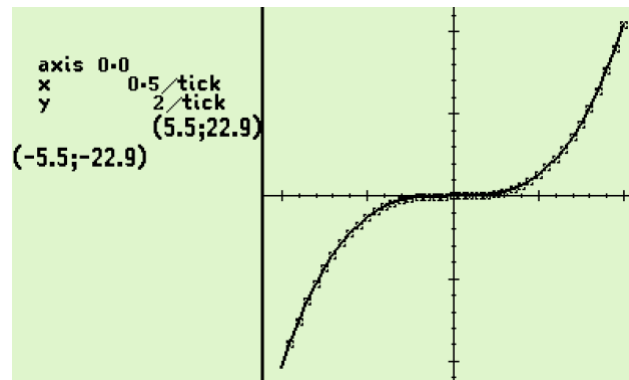


Figure 10b. A clean cubic $y^3/6$ through the origin: a double integral.

The result: Each of the plotted points (about 40, and up to around 80 where the curve steepens at the edges) is itself a completed double integral, worked out to full accuracy because the graph speed-up never touches the integrator. The curve is the textbook cubic, confirming that a nested integral behaves exactly like an ordinary function when you hand it to the grapher.

7. SOLVE of an INTEGRAL: the classic (SLVINT)

The question: Find the point x at which the running area under $g(t) = t$ first reaches 2. This is the gentle first taste of nesting: an integral inside a solve.

The formula: Solve $\int_0^x t dt - 2 = \frac{x^2}{2} - 2 = 0$. The answer is $x = 2$, since $\frac{2^2}{2} = 2$.

The program SI is the function “running area minus 2”, reusing HT. SLVINT solves SI for x .

```
LBL 'SI'               ; SI(x) = INT(0..x) t dt - 2 (reuses HT)
MVAR 'x'
PGMINT 'HT'
0 RCL 'x'
∫^x_y 't'              ; INT(0..x) t dt = x^2/2
2 -                    ; - 2
RTN

LBL 'SLVINT'           ; solve SI(x) = 0 -> x = 2
PGMSLV 'SI'
0 5                   ; guesses for x: lo -> Y, hi -> X
SOLVE 'x'
RTN
```

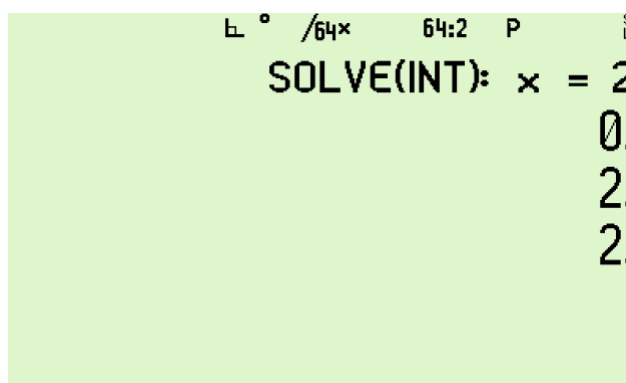


Figure 11a. The solve lands on $x = 2$. The answer is in X ; the solver's previous trial value sits in Y .

The result, to full width:

Register	Value as captured (34 significant figures)
X (root)	2.
Y (previous trial)	1.9999999999999999999999990615990431

The solver settled on exactly 2. The value in Y is Brent's method's second-to-last guess, already agreeing with the answer to about twenty-two figures, which shows how fast the search closed in. Because this solve stood on its own rather than inside a graph, it ran at full precision, not the reduced graph precision of example 1.

8. SOLVE of a SOLVE: a solve inside a solve (SLVSLV)

The question: For which value of p does the root of $f(x) = x^2 - p \cdot x - 2$ land exactly at 2? Answering it needs one solve inside another.

The formula: The outer solve looks for the p that makes $[\text{root of } f] - 2 = 0$. Setting $x = 2$ in f gives $4 - 2p - 2 = 0$, so $p = 1$.

The program EQ runs the inner solve RT (from example 1) to get the root, then subtracts
2. SLVSLV solves EQ for p.

```
LBL 'EQ' ; EQ(p) = root(p) - 2 (inner root from RT)
  XEQ 'RT' ; inner solve -> root x in X
  2 -
RTN

LBL 'SLVSLV' ; solve EQ(p) = 0 -> p = 1
  PGMSLV 'EQ'
  -5 5 ; guesses for p: lo -> Y, hi -> X
  SOLVE 'p'
RTN
```

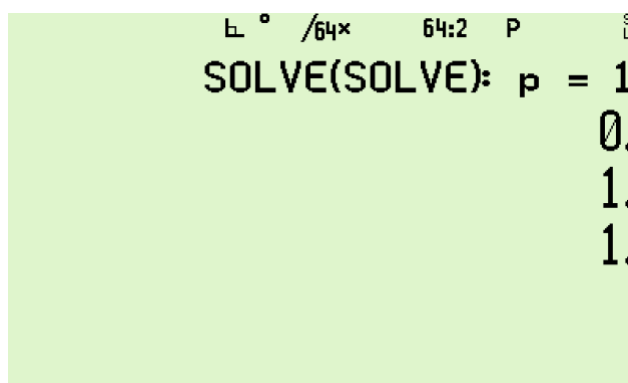



Figure 12a. The outer solve returns $p = 1$. The inner solve owned x ; the outer owned p , so the two never collided.

The result, to full width:

Register	Value as captured (34 significant figures)
X (root, p)	0.9999999999999999999999999999998
Y (previous trial)	0.99999999999999999999999999999991

The answer is 1 to within two parts in the thirty-fourth figure, the tiniest rounding at the very last digit. Both solves ran at full precision, and the only rule that mattered was that they solved for different variables: the inner search owned x , the outer owned p .

9. Standalone SOLVE at full precision: $\sqrt{2}$ (SLVF2)

The question: With nothing nested, how precisely can the solver find a root? The test is the square root of 2.

The formula: Solve $x^2 - 2 = 0$. The answer is $\sqrt{2} = 1.414213562373095048801668872420970...$

The program F2 is $x^2 - 2$. SLVF2 solves it, then also plots it so you can see the two crossings.

```
LBL 'F2' ; F2(x) = x^2 - 2
  MVAR 'x'
  RCL 'x' ENTER x
  2 -
RTN

LBL 'SLVF2' ; solve x^2 - 2 = 0 -> sqrt(2)
  PGMSLV 'F2'
  0 2 ; guesses lo -> Y, hi -> X
  SOLVE 'x'
  PGMPLOT 'F2'
  -2 4
  PLTf 'x' ; then draw the parabola
RTN
```

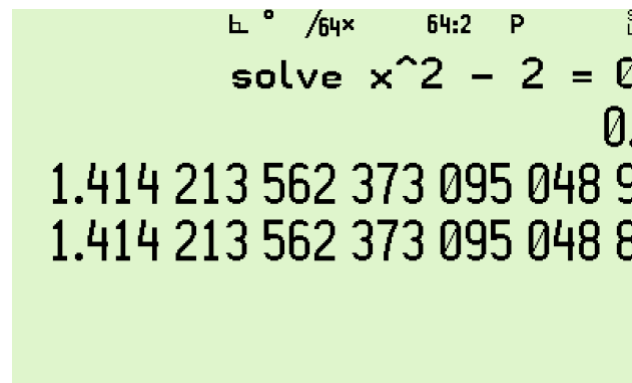


Figure 13a. $\sqrt{2}$ to the full width of the display. X holds the root; Y holds the previous trial, matching to about nineteen figures.

The result, to full width:

Register	Value as captured (34 significant figures)
X (root)	1.414213562373095048801688724209698
Y (previous trial)	1.414213562373095048798131937933543
True $\sqrt{2}$	1.41421356237309504880168872420969807856967... (reference)

The root matches the true $\sqrt{2}$ to the full thirty-four figures, the standalone precision of the calculator.

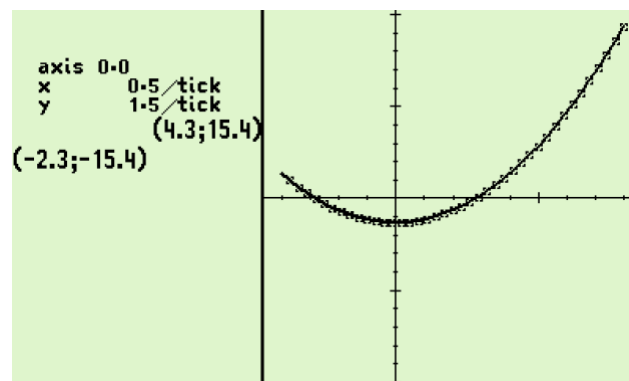


Figure 14b. The parabola $x^2 - 2$, crossing zero at $\pm\sqrt{2}$. The solver found the right-hand crossing.

10. Standalone INTEGRAL at full precision: π (INTPI)

The question: With nothing nested, how precisely can the integrator find an area? The test is a famous integral that equals π .

The formula: $\int_0^1 \frac{4}{1+x^2} dx = \pi = 3.14159265358979323846264338327950...$ because the integrand is the derivative of $4 \cdot \arctan(x)$, and $4 \cdot \arctan(1) = \pi$.

The program. FP is $\frac{4}{1+x^2}$. INTPI integrates it from 0 to 1. Note that the RAD setting is not required, as the integral is worked out numerically, not with trigonometry and angles. The

```

LBL 'FP' ; FP(x) = 4 / (1 + x^2)
MVAR 'x'
4
RCL 'x' ENTER * ; x^2
1 + ; 1 + x^2
÷ ; 4 / (1 + x^2)
RTN

LBL 'INTPI' ; INT(0..1) 4/(1+x^2) dx = pi
PGMINT 'FP'
0 1
∫^x_y 'x'
RTN

```

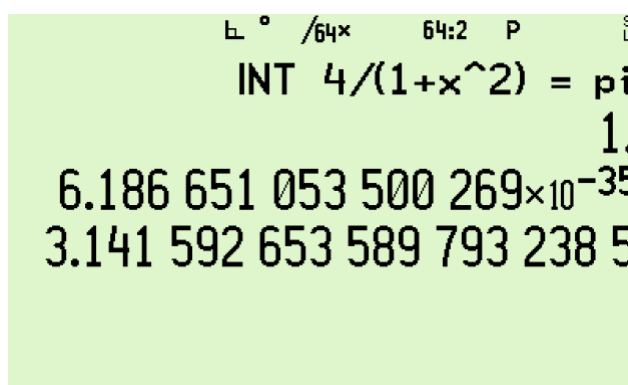


Figure 15. Δ in X , with the integrator's honest error estimate in Y (a few parts in 10^{35}) and the upper limit 1 in Z .

[illegible]

J Mostert © 2026

Closing Notes

Across the eleven ‘gauntlet’ integrals P01–P11, the C47/R47’s RPN-programmed $\int f$ integrator returned full or near-full precision on most problems: P01, P02, P04, P05, P06, P07 and P09 each agreed with the reference values to roughly 30–34 significant digits, with P07 exact. This places the 47 alongside the high-precision models, and ahead of every fixed-precision pocket calculator in the comparison, many of which returned ERR on the harder integrands.

Some of the ‘gauntlet’ integrals exposed limits of the built-in integrator; others are simply pathological by nature, hard for any method. P03 and P08 dropped to about 5 and 17 correct digits respectively, while P10 (accurate to ~2 digits) and P11 (accurate to ~1.6 digits) were the weakest albeit for pathological cases. For P11 the 47 returned 0.40096 against a reference of 0.39099. The deeper warning: the integrator’s own error estimate did not flag this shortfall, so the wrong value still looks precise. These are two different problems. P03 has interior corners, where $|x^2-3x|$ is not smooth (at $x = 0$ and $x = 3$); the remedy is to split the integral at those corners, not to tighten the accuracy. Splitting a range into smooth, manageable pieces is a normal, everyday part of numerical integration, not a workaround. P10 and P11 instead oscillate too violently for any fixed set of sample points to follow.

You also control the balance between speed and precision. SDIGS sets how hard the solver and grapher work; ACC sets the accuracy target for the integrator. The defaults aim for precision, which is usually what you want. If a job is too slow, loosen just the setting that controls it: SDIGS for a solve or plot, ACC for an integral.

The embedded examples showed that solve, integrate and plot are not three separate tools but building blocks that nest inside one another. Once one is allowed to sit inside another, it opens up problems a single operation cannot answer: a double or triple integral, the point at which a growing area first reaches a target, the parameter value that places a root exactly where you want it, or the path a root traces as that parameter changes. The 47 handled them all to the full thirty-four figures, not dropping accuracy for the examples tested. Nesting does have limits: Speed or lack thereof is the first. Then each inner embedded function must return a single number, so a plot cannot sit inside another function, and an integral cannot have a solve as its integrand.

In short, the RPN-programmed route delivers reference-grade accuracy on well-behaved integrals at the cost of a few seconds to roughly 100 s of run time on hardware, and degrades predictably on pathological integrands rather than failing outright.

Because of the same MVAR-based user function that drives `SOLVE`, $\int f$ and `PLTf`, a single program can serve for root-finding, integration and plotting alike.

Annex A: R47 rejig Program Listing

```

LBL 'INTDEMO'
RAD
SF 'SSIZE8'

11          * Prep: loop from 11 to 1
STO R00
LBL 00

* == LOOP START ==
' BND_P'    * Prep: indirect bounds
RCL R00
+
STO R02

'INT_P'     * Prep: indirect integr.
RCL R00
+
STO R01

* == SET AND DRAW GRAPH ==
XEQ →R02    * Set graph bounds
PLTRST     * Reset graph settings
PGMPLT →01 * Set graph RPN eq.
PLTF 'x'   * Do RPN graph plot(x)
SNAP
PAUSE 10
* USE BYPASS TO SKIP INTEGRATION
* GTO 'BYPASS'

* == SET AND INTEGRATE ==
XEQ →R02    * Set integral bounds
PGMINT →R01 * Set integration eq.
∫^x_y 'x'   * Do integration
LASTT?     * Push integration time

RCL Y       * Get result
RCL R01     * Get function number
⇨xy        * Output #No. & Result

R↓ R↓      * Roll down time & acc.
10 ÷ 's' +  * Get seconds
RCL Z       * Get declared accuracy
⇨xy        * Output accuracy & time

RCL Z       * Get declared accuracy
÷ |x| log10(x) CHS
'Relative Accuracy:'
⇨xy        * Output accuracy

LBL 'BYPASS'
* == END LOOP ==
DSZ R00
GTO 00
RTN

LBL 'BND_P1'
1
2
RTN

LBL 'INT_P1'
* f(x) = 1/√x,
* bounds [1, 2]
MVAR 'x'
RCL 'x'
√x 1/x
RTN

```

```

LBL 'BND_P2'
-1
3
RTN

LBL 'INT_P2'
* f(x) = e^(-x^2),
* bounds [-1, 3]
MVAR 'x'
RCL 'x'
x^2 CHS e^x
RTN

LBL 'BND_P3'
-15
15
RTN

LBL 'INT_P3'
* f(x) = |x^2 - 3x|,
* bounds [-15, 15]
MVAR 'x'
RCL 'x' ENTER
x^2 x↔y 3 * - |x|
RTN

LBL 'BND_P4'
0
1.57079
RTN

LBL 'INT_P4'
* f(x) = tan(x),
* bounds [0, 1.57079]
MVAR 'x'
RCL 'x' tan(x)
RTN

LBL 'BND_P5'
0
1
RTN

LBL 'INT_P5'
* f(x) = x^(x^(x^x)),
* bounds [0, 1]
MVAR 'x'
RCL 'x'
ENTER ENTER ENTER
y^x y^x y^x
RTN

LBL 'BND_P6'
0
1
RTN

LBL 'INT_P6'
* f(x) = cos(x)·ln(x),
* bounds [0, 1]
MVAR 'x'
RCL 'x'
cos(x)
RCL 'x'
ln(x)
*
RTN

```

```

LBL 'BND_P7'
0
1
RTN

LBL 'INT_P7'
* f(x) = 1/√x,
* bounds [0, 1]
MVAR 'x'
RCL 'x' √x 1/x
RTN

LBL 'BND_P8'
0
1
RTN

LBL 'INT_P8'
* f(x) = 1/√(-ln(x)),
* bounds [0, 1]
MVAR 'x'
RCL 'x' ln(x) CHS √x 1/x
RTN

LBL 'BND_P9'
0
1
RTN

LBL 'INT_P9'
* f(x) = ln(Γ(x))
* bounds [0, 1]
MVAR 'x'
RCL 'x' Γ(x) ln(x)
RTN

LBL 'BND_P10'
-1
1
RTN

LBL 'INT_P10'
* f(x) = e^(x+sin(e^e^e^(x+1/3)))
* bounds [-1, 1]
MVAR 'x'
RCL 'x' 1 3 / +
e^x e^x e^x SIN
RCL 'x' +
e^x
RTN

LBL 'BND_P11'
0
1
RTN

LBL 'INT_P11'
* f(x) = sin^2(tan(tan(x)))
* bounds [0, 1]
MVAR 'x'
RCL 'x' π * tan(x) tan(x) sin(x) x^2
RTN

.END.

```

Process: Above programs were written in the 47 editor, saved, converted by [rejig](#) to plain text. Comments were added in free form text, re-converted back to 47.

Annex B: Alternative graphing: PLSTAT

The C47/R47 calculator has a dedicated plotting function for user RPN programs in addition to the EQN menu designed to plot algebraic formulas.

Plot (x, y) graphs using the STAT and PLOT menus

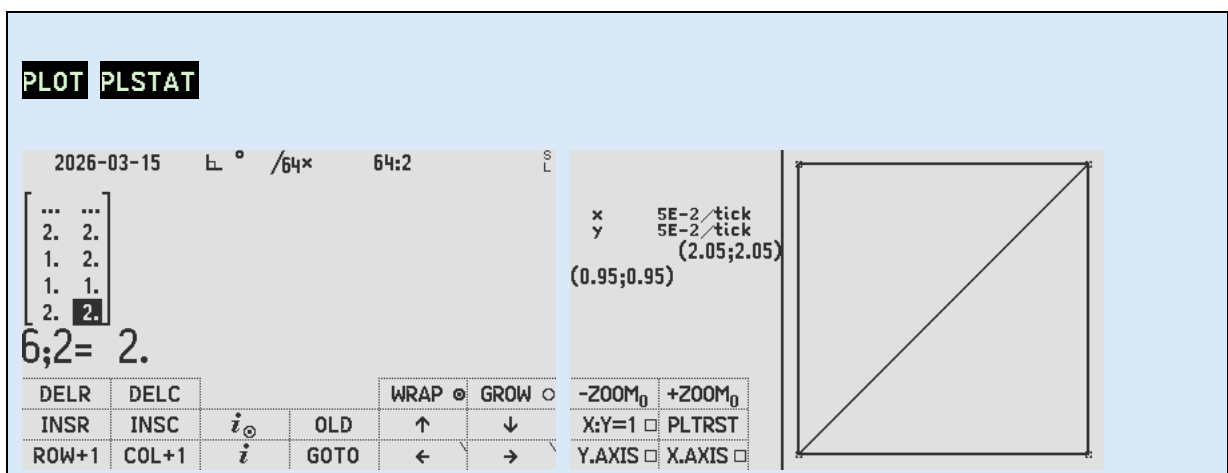
This method allows you to store a matrix of points via the $\Sigma+$ entry system; the concept is to store coordinates and connect the dots with lines. This method is employed in older demo programs such as BinetV4.p47.

To plot a graph, use the 47 statistical functions. This involves entering (x,y) pairs into the STATS matrix. You can fill this matrix using the $\Sigma+$ or M.EDIT functions. Once the matrix is filled with coordinates, you can plot the graph using PLSTAT, which will draw lines between the points.

To manually plot from the STATS matrix, create a 6x2 matrix and populate it with a graph resembling a square and diagonal. Follow the example below carefully, entering the keypresses as you go.

```
CLS MATX 6 ENTER 2 NEW
EDIT 1 → 1 →    2 → 1 →    2 → 2 →    1 → 2    1 → 1    2 → 2
EXIT STO α 'STATS'
```

Edit a new matrix, exit the editor, store to STATS, and plot using PLSTAT



Plot using PLSTAT

Graph an RPN function using PLSTAT:

For an RPN function, the PLSTAT statistics system can be utilised to generate a graphical representation. Employing the concept outlined earlier regarding plotting coordinates, execute a loop to iterate over varying values of x for screen resolution, spanning from 0 to 399. Subsequently, determine the x-domain, potentially extending from say 0 to 2π . Calculate the corresponding y-values, denoted as $f(x)$, using the RPN function. Convert

the y-domain, which ranges from say 0 to 1, to a screen resolution that spans from 0 to 240, considering that the y-axis is inverted, and 0 represents the top line.

For illustrative purposes, load the provided BinetV4.p47 program (located within the PROGRAMS folder of the distribution zip file) to observe how this process can be executed in a manner analogous to the aforementioned steps. Specifically, calculate and populate the graph coordinates into the STATS matrix, and subsequently plot the resulting data. Below is an example of how to modify the Binet program to plot the FUNC function discussed in the preceding sections.

Graphing the prior RPN function 'FUNC' using the STAT and PLOT menus

As an easy start, this appnote guides you to load up the **BinetV4.p47** program (in the PROGRAMS folder in the distro zip file) and minimally modify that program to plot your 'FUNC' program: Changes to the Binet program are in yellow, with removals and changes as indicated. Grey text below is unchanged from referenced program BinetV4.p47:

C47 Program file export: Export format version 3, C47 program version 1.

0000: Prgm #43: 171 bytes / 55 steps

0001: LBL 'Binet PLOTABC'

0002: REM 'Plot the FUNC function, V04.

0003: LocR 11

0004: STOCFG .02

0005: STOS .03

0006: CLΣ

0007: 101

0008: STO .00

0009: STO .01

0010: LBL B

0011: RCL .01

0012: RCL .00

0013: -

0014: RCL .01

0015: XEQ A

0016: AVIEW .00

0017: Σ+

0018: DSZ .00

0019: GTO B

0020: RCLS .03

0021: RCLCFG .02

0022: 'PRESS R/S TO PLOT'

0023: STO 00

0024: DROPx

0025: PROMPT 00

0026: PLSTAT

0027: PLTFCNS

0028: RTN

```

0029: LBL A
0030: REM 'CALC P1'
0031: DECR X
0032: ÷

0033: 7 // Change: to 6, being (3-(-3))
0034: ×
0035: 2 // Change: to 3, being the negative graph offset
0036: -
0037: ENTER STO 'x' // Add: store the new x value
0038: # 74 φ r.golden XEQ 'FUNC' // Call your RPN math function
0039: ×>y RCL 'x' // Push the x-coordinate to X
0040: y×
0041: ×>y
0042: # 74 φ r.golden
0043: CHS
0044: ×>y
0045: CHS
0046: y×
0047:
0048: 5
0049: √x
0050: +
0051: ENTER
0052: Im
0053: ×>y
0054: Re
0055: RTN
0056: .END.

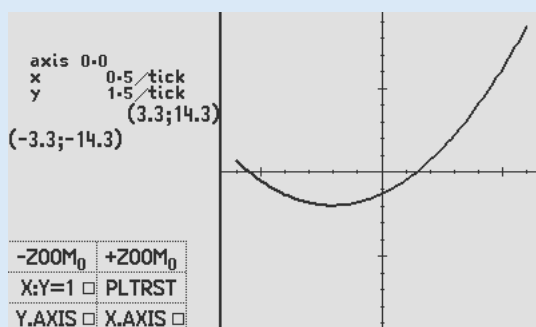
```

Modified BinetV4.p47 program

Plot it

Get the graphic result by typing:

XEQ α PLOTABC ENTER to run the RPN program sequencing the plot below:



Pro tip 4:

PLTRST (Plot Reset) will reset the view settings, so feel free to experiment.

Output plot of the new PLOTABC RPN program

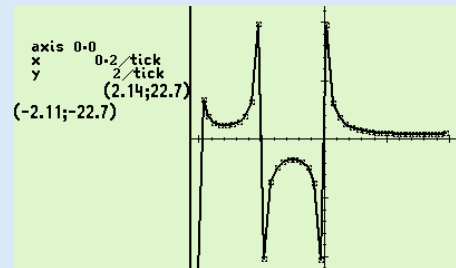
- Once the **STATS** matrix exists, it can be copied to any register for later view.
Example: use **RCL α STATS STO 00**
- You can **RCL** a stored matrix, **STO** it as the **STATS** matrix, and view using **PLSTAT**.
Example: use **RCL 00 STO α STATS**
- Once a matrix is stored into **STATS**, all accumulators are re-calculated, after which **PLSTAT** is available to use.

Annex C: Controlling graph parameters

You can pre-set the graph parameters before PLTf. See example below:

```

LBL 'GRAF-G'
  PGMPLOT 'FUNC'
  -2
  2
  PLTRST      # Resets to default settings (find op P.FN1)
  20.
  STO '↑Y'    # Y upper value
  CHS
  STO '↓Y'    # Y lower value
  0
  ZOOM        # Set zoom factor, example shows 2
  CF PLnvec   # Set vector plotting mode (nav) with N = 0° clockwise
  CF PLvec    # Set vector plotting mode (math)
  CF PLxy:1   # Set symmetrical scales on x and y to be 1:1
  SF PLx.ax   # Set y-axis to be on screen
  SF PLy.ax   # Set x-axis to be on screen
  CF PLplus   # Set point marker to a plus
  CF CFcross  # Set point marker to cross
  SF PLbox    # Set point marker to box
  CF PLcurve  # (disabled)
  CF PLcxp    # Plot complex plot
  CF PLimp    # Plot imaginary part
  PLTf 'x'
  SNAP
  RTN
  LBL 'GAMMA'
  MVAR 'x'
  RCL 'x'
  Γ(x)
  RTN
  
```



Comparative results: Source Duncan Murray, SwissMicros Forum

calculator	P01	P02
Ref	0.8284271247461900976033774484193961571393	1.6330314810719482592939520140436985849241
WP43	0.8284271247461900976033774484193961	1.633031481071948259293952014043699
Casio fx-115W	0.82843	1.633
Sharp EL-W506T	0.828427124	1.633031482
TI-83 Plus	0.8284271247	1.633031481
Casio fx-9910CW		
TI-NSpire CX II KhiCAS 2.0		
HP Prime 2		
TI-89 Ti		
Numworks 24.10.0	0.82842712474619	1.6330314810719
R47	0.8284271247461900976033774484193961	1.633031481071948259293952014043699
TI-NSpire CX II		
DM-15L FIX 4	0.8284	1.6330
db48x 0.9.14 S12	0.828442760473	1.63303136781
db48x 0.9.14 S24a	0.828426601063264722661203	1.63303136786981098140461
db48x 0.9.14 S24b	0.828427124746188196214920	1.63303148107194826514048
db48x 0.9.14 S48	0.828427124746190097982799267593872004769254	1.63303148107194826514073710458322273269153
db48x 0.9.14 Sstd	0.828427124746190097982713	1.63303148107194826514048
HP-34C SCI 9	0.8284271248	
DM-15L SCI 8	0.8284271248	1.633031481
Casio fx-991EX	0.82842712474618	1.63303148110593
calculator	P03	P04
Ref	2259	11.9707167854931391832566150503243090924474
WP43	2258.993388206803467294972568816453	11.97071678549313918325661505033089
Casio fx-115W	2250	ERR
Sharp EL-W506T	2259	419.4464408
TI-83 Plus	2250	11.97071678
Casio fx-9910CW	2259	11.9707167854931
TI-NSpire CX II KhiCAS 2.0		11.9707167841
HP Prime 2		11.9707167841
TI-89 Ti	2259	11.970716784959
Numworks 24.10.0	2259	11.970716785491
R47	2258.993388206803467294972568816451	11.97071678549313918325661505033089
TI-NSpire CX II	2250	
DM-15L FIX 4	2258.9998	11.9707
db48x 0.9.14 S12	2259.11276851	11.9706034618
db48x 0.9.14 S24a	2259.00218853262525219728	11.9707171781668250584788
db48x 0.9.14 S24b	2258.99999999892401571912	11.9707167854931447921917
db48x 0.9.14 S48	ERR	11.9707167854931391826767452455061658964765
db48x 0.9.14 Sstd	ERR	ERR
DM-15L SCI 8	2259.000006	ERR
Casio fx-991EX	2250	11.9707167841225

calculator	P05	P06
Ref	0.7313397798915709984937218416521334207830	-0.9460830703671830149413533138231796578123
WP43	0.7313397798915709984937218416521334	-0.946083070367183014941353313823179
Casio fx-115W	ERR	ERR
Sharp EL-W506T	ERR	ERR
TI-83 Plus	0.7313394496	-0.9460814174
Casio fx-9910CW		
TI-NSpire CX II KhiCAS 2.0		
HP Prime 2		
TI-89 Ti		
Numworks 24.10.0	0.73133977989157	-0.94608307036718
R47	0.7313397798915709984937218416521334	-0.9460830703671830149413533138231796
TI-NSpire CX II		
DM-15L FIX 4	0.7313	-1.0000
db48x 0.9.14 S12	0.940522403160	-0.945912828691
db48x 0.9.14 S24a	0.940318095892148559294545	-0.946072479866791647174107
db48x 0.9.14 S24b	0.940318086681907169857535	-0.946083070367025410749963
db48x 0.9.14 S48	0.940318086681907169857608447182094166044772	ERR
db48x 0.9.14 Sstd	0.940318086681907169857535	ERR
HP-34C SCI 9		
DM-15L SCI 8	0.7313397799	-0.9460830688
Casio fx-991EX	ERR	ERR
calculator	P07	P08
Ref	2	1.7724538509055160272981674833411451827975
WP43	2	1.772453850905516013219...
Casio fx-115W	ERR	ERR
Sharp EL-W506T	ERR	ERR
TI-83 Plus	1.999994424	1.772446281
Casio fx-9910CW		
TI-NSpire CX II KhiCAS 2.0		
HP Prime 2		
TI-89 Ti		
Numworks 24.10.0	2	1.7724538406522
R47	2	1.772453850905516013219002902374754
TI-NSpire CX II		
DM-15L FIX 4	1.9999	1.772324417
db48x 0.9.14 S12	1.99896964912	1.77142350570
db48x 0.9.14 S24a	1.99987120626587583845160	1.77232505722451111834911
db48x 0.9.14 S24b	ERR	ERR
db48x 0.9.14 S48	ERR	ERR
db48x 0.9.14 Sstd	ERR	ERR
HP-34C SCI 9		
DM-15L SCI 8	ERR	ERR
Casio fx-991EX	ERR	ERR
calculator	P09	P10
Ref	0.9189385332046727417803297364056176398613	2.996339811976535379182861556235501887668
WP43	0.9189385332046727417803297364056176	2.975...
Casio fx-115W	ERR	ERR

Sharp EL-W506T	ERR	ERR
TI-83 Plus	ERR	ERR
Casio fx-9910CW		
TI-NSpire CX II KhiCAS 2.0		
HP Prime 2		
TI-89 Ti		
Numworks 24.10.0	ERR	ERR
R47	0.9189385332046727417803297364056176	2.969857419600580236632783812721218
TI-NSpire CX II		
DM-15L FIX 4	0.9189	3.002381936
db48x 0.9.14 S12	0.918927942647	ERR
db48x 0.9.14 S24a	0.918938530622447629129603	ERR
db48x 0.9.14 S24b	ERR	ERR
db48x 0.9.14 S48	ERR	ERR
db48x 0.9.14 Sstd	ERR	ERR
DM-15L SCI 8	0.9189385317	ERR
Casio fx-991EX	ERR	ERR
calculator	P11	
-----	-----	
Ref	0.3909921621515304545317098808049744853103	
WP43	0.398...	
Casio fx-115W	0	
Sharp EL-W506T	ERR	
TI-83 Plus	ERR	
Casio fx-9910CW		
TI-NSpire CX II KhiCAS 2.0		
HP Prime 2		
TI-89 Ti		
Numworks 24.10.0	0.35816045512655	
R47	0.4009624732903700675809525961653749	
db48x 0.9.14 (prec 42)		
TI-NSpire CX II		
DM-15L FIX 1	0.4169042624	
DM-15L FIX 2	0.3916722662	
DM-15L FIX 3	0.3955440737	
DM-15L FIX 4	0.3911292357	
DM-15L SCI 8	ERR	
db48x 0.9.14 S12	ERR	
db48x 0.9.14 S24a	ERR	
db48x 0.9.14 S24b	ERR	
db48x 0.9.14 S48	ERR	
db48x 0.9.14 Sstd	ERR	
Casio fx-991EX	ERR	

Author Jaco Mostert; proofread by Duncan Murray; assisted by Claude

Copyright (C) 2026 The C47/R47 Development Team. Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license can be downloaded from [gnu.org](https://www.gnu.org).